



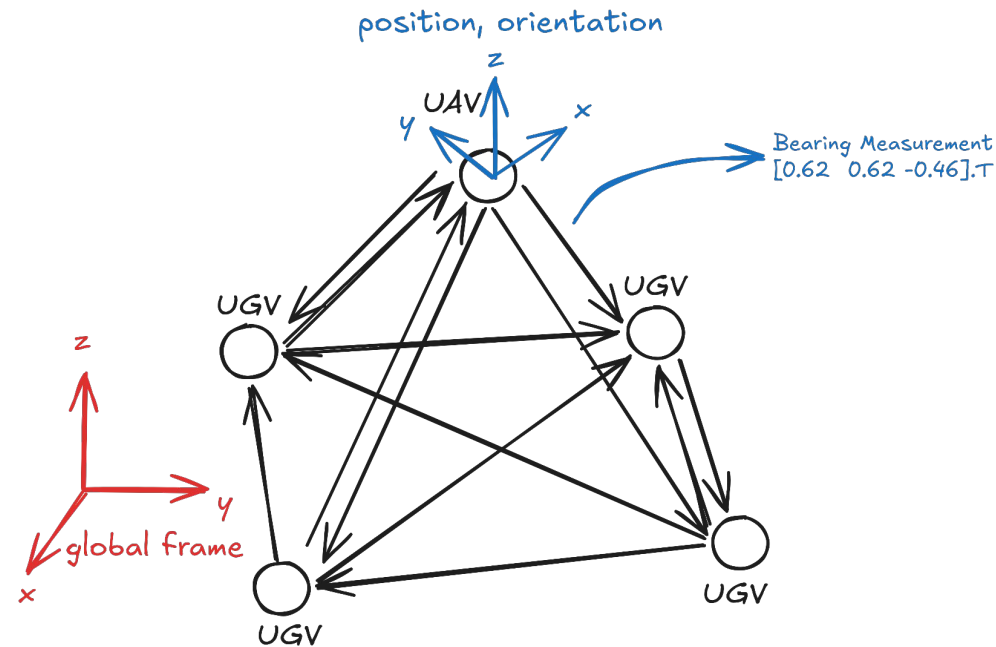
UNIVERSITÀ
DI PADOVA

Network Topology Optimization for Bearing Rigid Multi-Agent Systems via Deep Reinforcement Learning and Graph Neural Networks

Noyan Erdin Kilic

https://github.com/nyanklc/rigidity_rl

04/08/2026



Bearing Rigidity

Given all bearings:
 $[x_0, y_0, z_0, x_1, \dots].T$



Can we tell the shape
 of the graph/framework
 (uniquely)?

(up to a
 translation/rotation/scale
 factor)

Bearing Rigidity Matrix

$$\dot{\mathbf{b}}_g(\chi(t)) = \frac{d}{dt} \mathbf{b}_g(\chi(t)) = \mathbf{B}_g(\chi(t)) \delta.$$

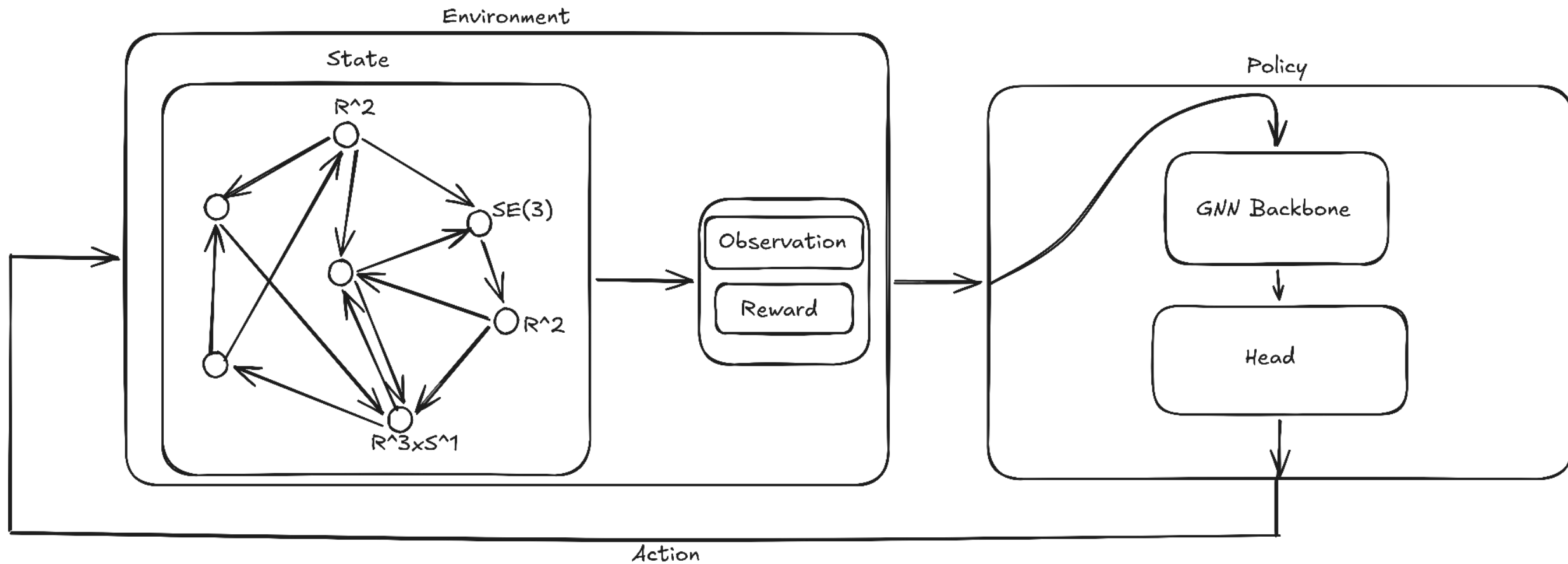
How to check for Infinitesimal
 Bearing Rigidity?

$$\text{rank}(\mathbf{BRM}_{\text{graph}}) == \text{rank}(\mathbf{BRM}_{\text{fullyconnected}})$$

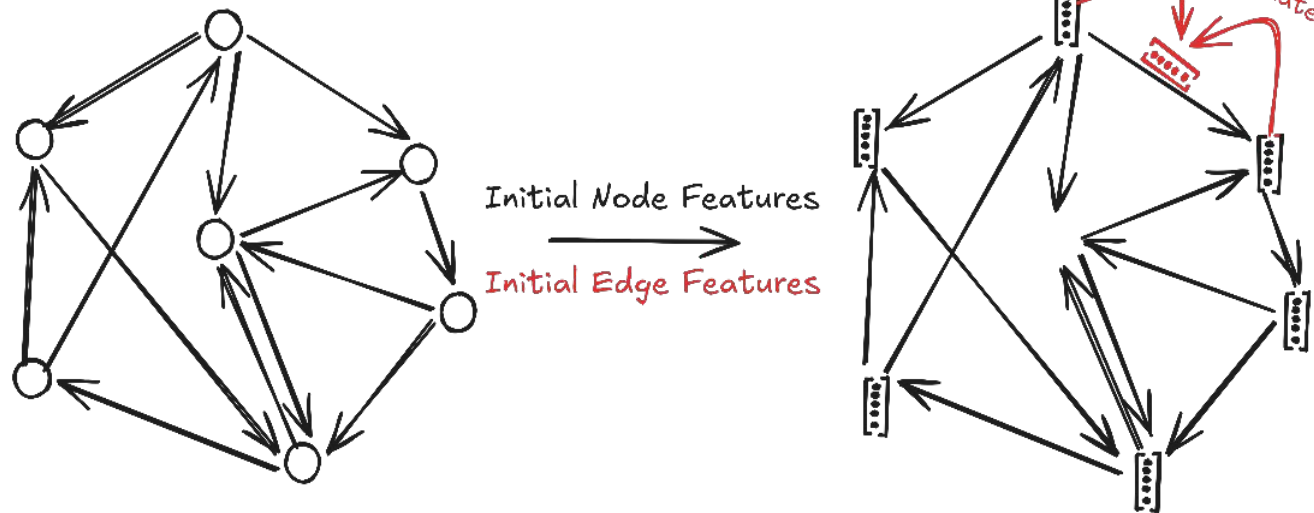
Some other tools at our disposal:

- Minimal Bearing Rigidity: We have a closed form solution to the minimum number of edges needed for a framework to be rigid. This only works on homogeneous networks with domain $\mathbb{R}^d$.
- (Heuristic) Generalized MBR: A greedy per-edge counting of the DOF constraints introduced by the bearing measurements. We can check if these constraints account for all non-trivial DOFs, and if any redundant edges remain. Estimates (on average) a lower number than the true value for heterogeneous networks.

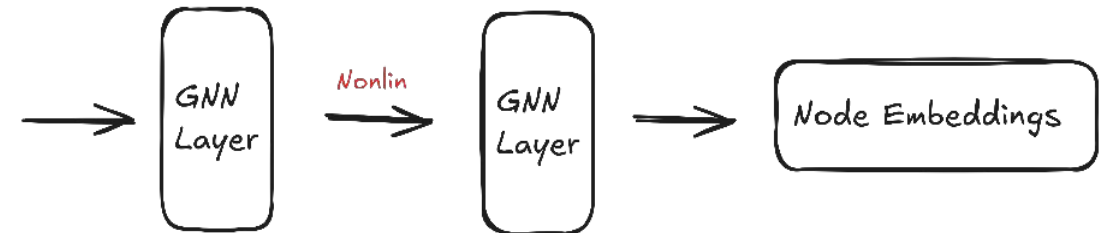
- Is there an optimal graph topology that balances sparsity and structural rigidity?
 - Can deep reinforcement learning be used to automatically construct such a topology?
 - What kind of an architecture can be used for the policy and how can it be trained?
 - Can a learned policy generalize to previously unseen network configurations?
 - What graph structures emerge, and what properties characterize them when optimized simultaneously for bearing rigidity and graph sparsity?
-



Message Passing Neural Networks



$$\mathbf{x}'_i = \gamma_{\Theta} \left(\mathbf{x}_i, \bigoplus_{j \in \mathcal{N}(i)} \phi_{\Theta}(\mathbf{x}_i, \mathbf{x}_j, \mathbf{e}_{j,i}) \right)$$



Graph Isomorphism Network

$$h_v^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \epsilon^{(k)} \right) \cdot h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right).$$

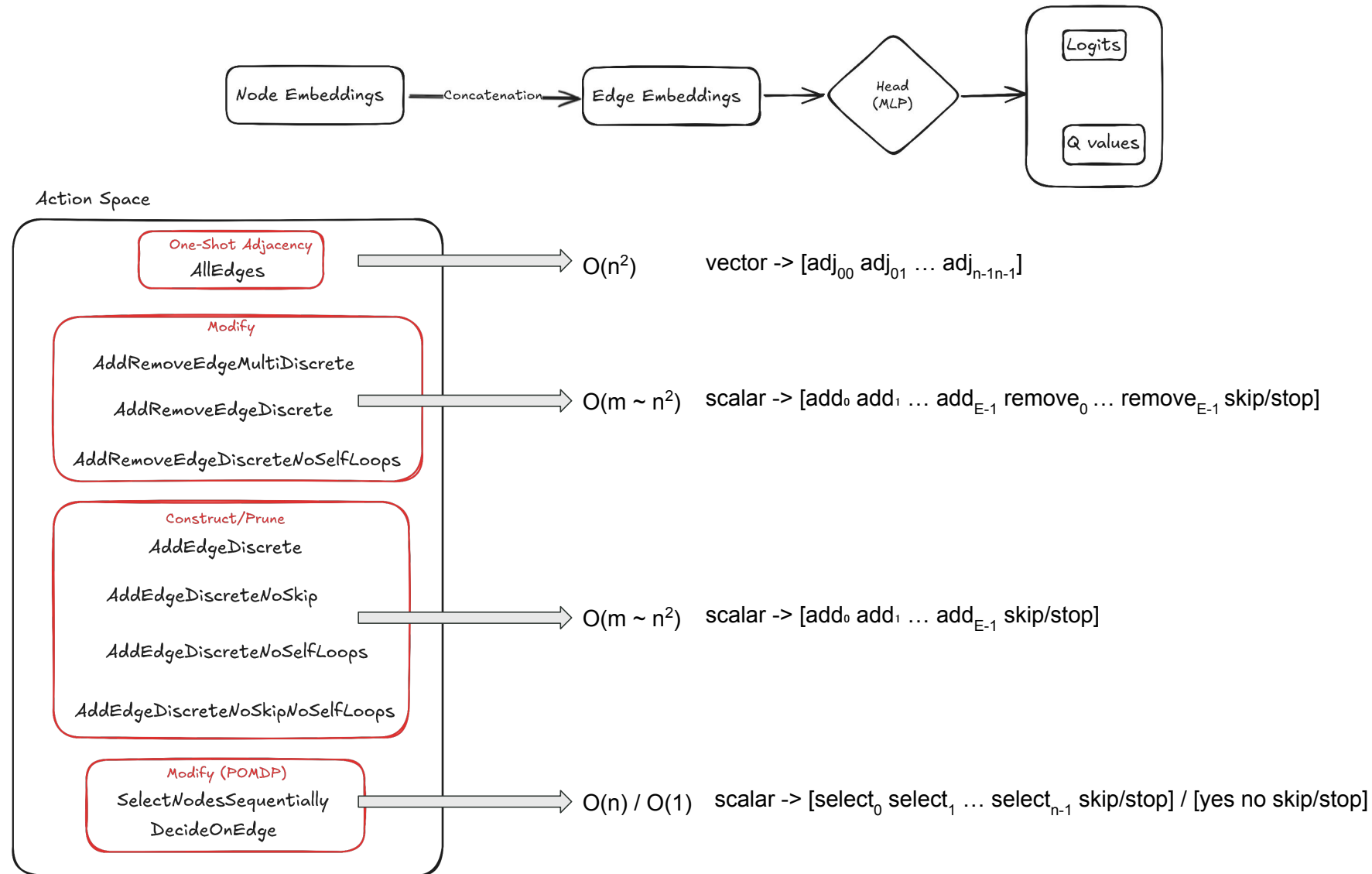
Equivariant Graph Neural Network

$$\mathbf{m}_{ij} = \phi_e \left(\mathbf{h}_i^l, \mathbf{h}_j^l, \|\mathbf{x}_i^l - \mathbf{x}_j^l\|^2, a_{ij} \right)$$

$$\mathbf{x}_i^{l+1} = \mathbf{x}_i^l + C \sum_{j \neq i} (\mathbf{x}_i^l - \mathbf{x}_j^l) \phi_x(\mathbf{m}_{ij})$$

$$\mathbf{m}_i = \sum_{j \neq i} \mathbf{m}_{ij}$$

$$\mathbf{h}_i^{l+1} = \phi_h(\mathbf{h}_i^l, \mathbf{m}_i)$$



Problems/Considerations:

- Absolute positions as node features -> GNNs can't generalize
- Absolute orientations as node features -> GNNs can't generalize
- Can't use bearings as node features -> Node feature size depends on degree
- We would like to use edge features for bearings -> GNNs are generally designed for node features
- We don't have much information in our state formulation (just poses and bearings) -> Need to derive information
- (?) Using non-existent edge/bearing information would defeat the purpose -> Can only use current edges' information

Observation

- **Node Features:** {Domain ($\mathbb{R}^2$, SE(3) etc.), In/Out Degree, Centralities (closeness, eigenvalue), Betweenness}
- **Edge Features:** {Bearing (3-dim vector, Euler angles), Betweenness, Reciprocity, Common Neighbors}
- **Adjacency**
- **(EGNN) Coordinates**
- **(Selection action) Current Selection**

Note: The graph theoretic node level features are not commonly used with graph neural networks since they have the ability to learn them implicitly.

State Score:

$$\text{score} = \overset{20}{w_{\text{rank}}} \cdot \text{rank}(B) - \overset{10}{w_{\text{edge}}} \cdot |E| + \overset{0}{w_{\text{ibr}}} \cdot \mathbb{I}_{\text{IBR}} + \overset{0}{w_{\text{eig}}} \cdot \mathbb{I}_{\text{IBR}} \cdot \ln(1 + \overset{0}{w_{\text{eig1}}} \cdot \lambda_{\text{rigidity}})$$

- How “good” the current state is
- Weights are hyperparameters
- Potential based shaping

Types of rewards (per step):

- State reward ($\text{score}_{\text{curr}} - \text{score}_{\text{prev}}$)
- Time penalty
- Action reward
- Termination reward

-> Bonuses when the termination condition is met or certain actions are taken.

Action	Reward
Add edge, rank +1 (2D measurer)	+10
Add edge, rank +2 (3D measurer)	+30
Add edge, rank +0	-10
Remove redundant edge	+10
Remove rigidity-critical edge	-10 (or -30)
Skip	0

```
# STATE_SCORE_TYPE = "Rigid"
# STATE_SCORE_TYPE = "RigidAndMinEigenvalue"
# STATE_SCORE_TYPE = "RigidAndMinRigid"
# STATE_SCORE_TYPE = "RigidAndLogMinEigenvalueAndEdges"
# STATE_SCORE_TYPE = "MinRigid"
# STATE_SCORE_TYPE = "MinRigidAndMinEigenvalue"
# STATE_SCORE_TYPE = "MinEigenvalue"
# STATE_SCORE_TYPE = "Eigenvalues"
# STATE_SCORE_TYPE = "EdgeCount"
# STATE_SCORE_TYPE = "LogMinEigenvalue"
# STATE_SCORE_TYPE = "RigidityMatrixRank"
# STATE_SCORE_TYPE = "RigidityMatrixRankAndEdges"
# STATE_SCORE_TYPE = "Weighted"
# STATE_SCORE_TYPE = "None"
```


Algorithms:

- PPO and DQN, both via skrl
- Actor / critic / Q-network selected automatically from (action space × observation type)
- Same GNN backbone in all cases; only the action head changes

One episode:

- Reset -> Random Graph
- Per step: GNN -> action head -> masked action -> apply change -> reward
- Terminate on condition

Action masking:

- Invalid actions masked inside the model ($-1e9$ on the logit/Q-value), not in the environment
 - DQN applies the same mask to e-greedy exploration
-

- Directed Erdős-Rényi Model $G(n, m)$
- Fixed number of nodes (and domains)
- Random positions and orientations
- **Random number of edges:**
 - If sampled uniformly over all possible numbers of edges, the expected value is: $(n^2 - n)/2$
 - However the minimum number of edges required for IBR grows roughly linearly with n .
 - This means for most episodes, the graph will need (possibly lots of) pruning. This makes learning construction actions difficult.
 - **Solution:** Sample uniformly (or normal dist.) centered around the approximate minimum number of edges for rigidity.

n	domain	E[m]	MBR needs	ratio
4	R^2	6.0	5	1.2×
8	R^2	28.0	13	2.2×
8	R^3	28.0	10	2.8×
12	R^3	66.0	16	4.1×

MBR:

- For homogeneous networks in $\mathbb{R}^d$, we have a closed form solution.
- For heterogeneous networks, or homogeneous in higher level domains, there is no closed form solution to the minimum number of edges required for bearing rigidity.
- **Heuristic (Lower Bound):** Count the DOFs each edge in the network constrains. The minimum number of edges required for bearing rigidity can be lower bounded by counting the total constrained DOFs greedily by sorting the edges in order of number of DOFs they constrain.

Theorem 4.1: The graph $\mathcal{G}$ of n ($n \geq 3$) vertices is MBR in $\mathbb{R}^d$ if and only if it is GBR in $\mathbb{R}^d$ and has exactly $f(n, d)$ edges, where $f(n, d) = n$, if $n \leq d + 1$, and

$$f(n, d) = 1 + \left\lfloor \frac{n-2}{d-1} \right\rfloor \times d + \text{mod}(n-2, d-1) + \text{sgn}(\text{mod}(n-2, d-1)) \quad (26)$$

if $n \geq d + 1$.

Max steps:

- Let the RL agent accumulate rewards as much as possible during the episode.
- Relying on the agent taking the “skip” action when it thinks it reached the optimal state.

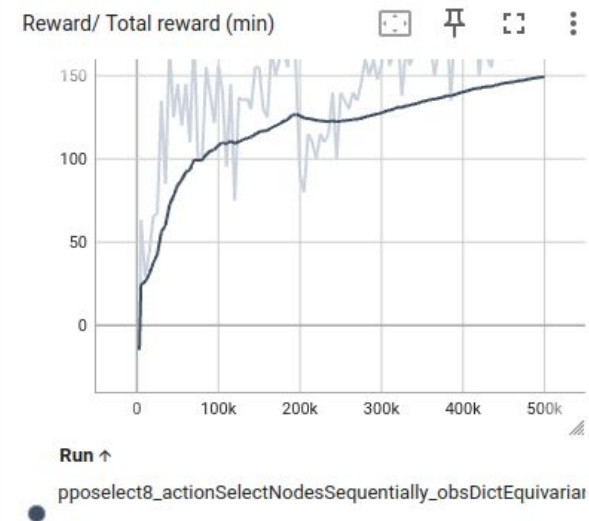
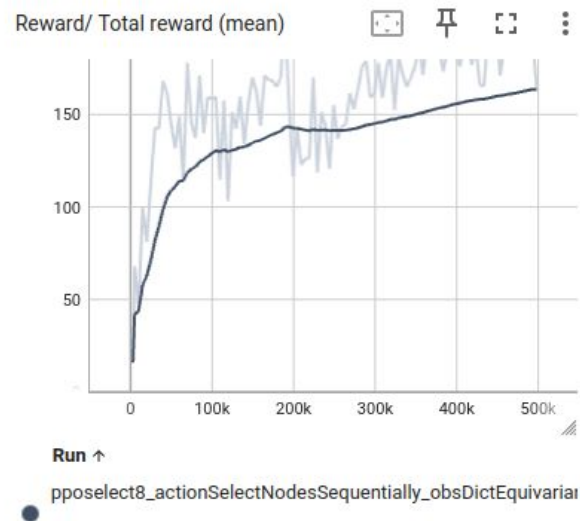
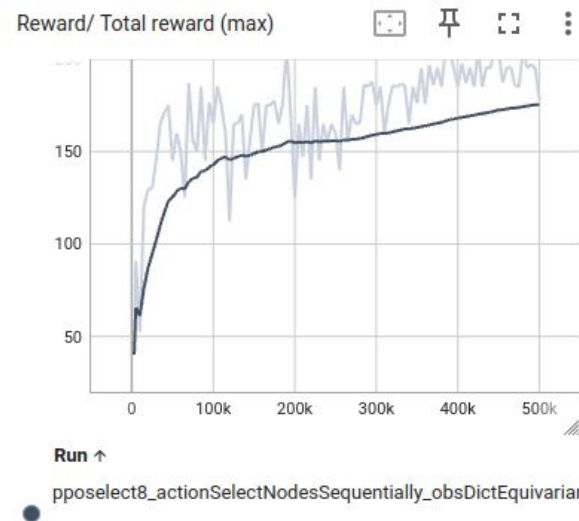
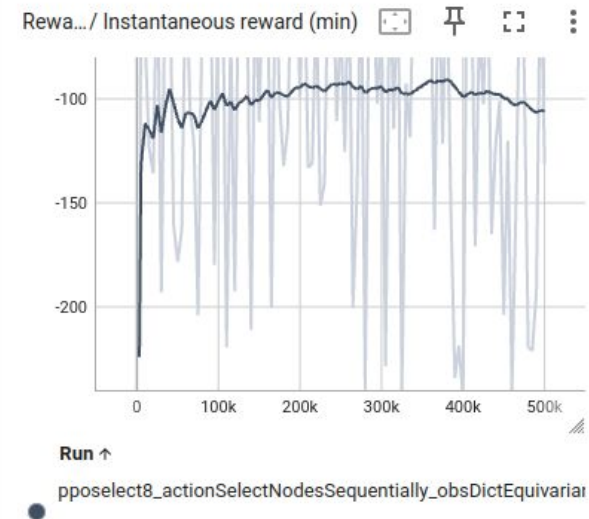
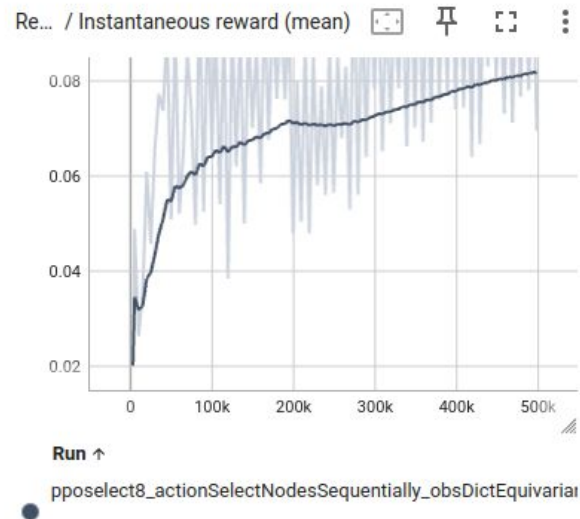
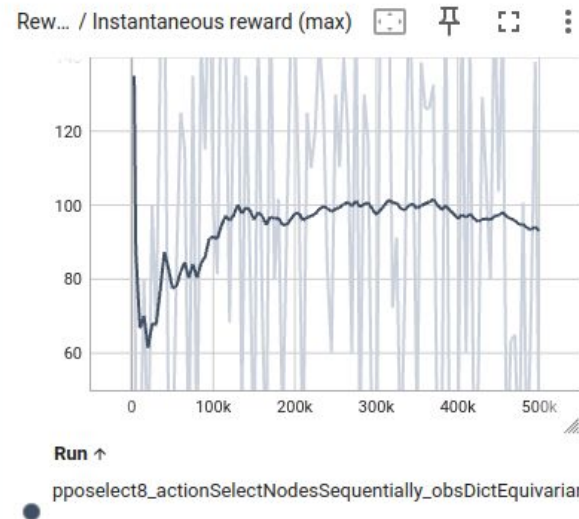
The policy is effectively an optimizer **over the possible configurations of edges.**

Trained on:

- 8 nodes in R^3
- “MaxSteps” termination

Model:

- PPO
- EGNN
- Select Nodes action

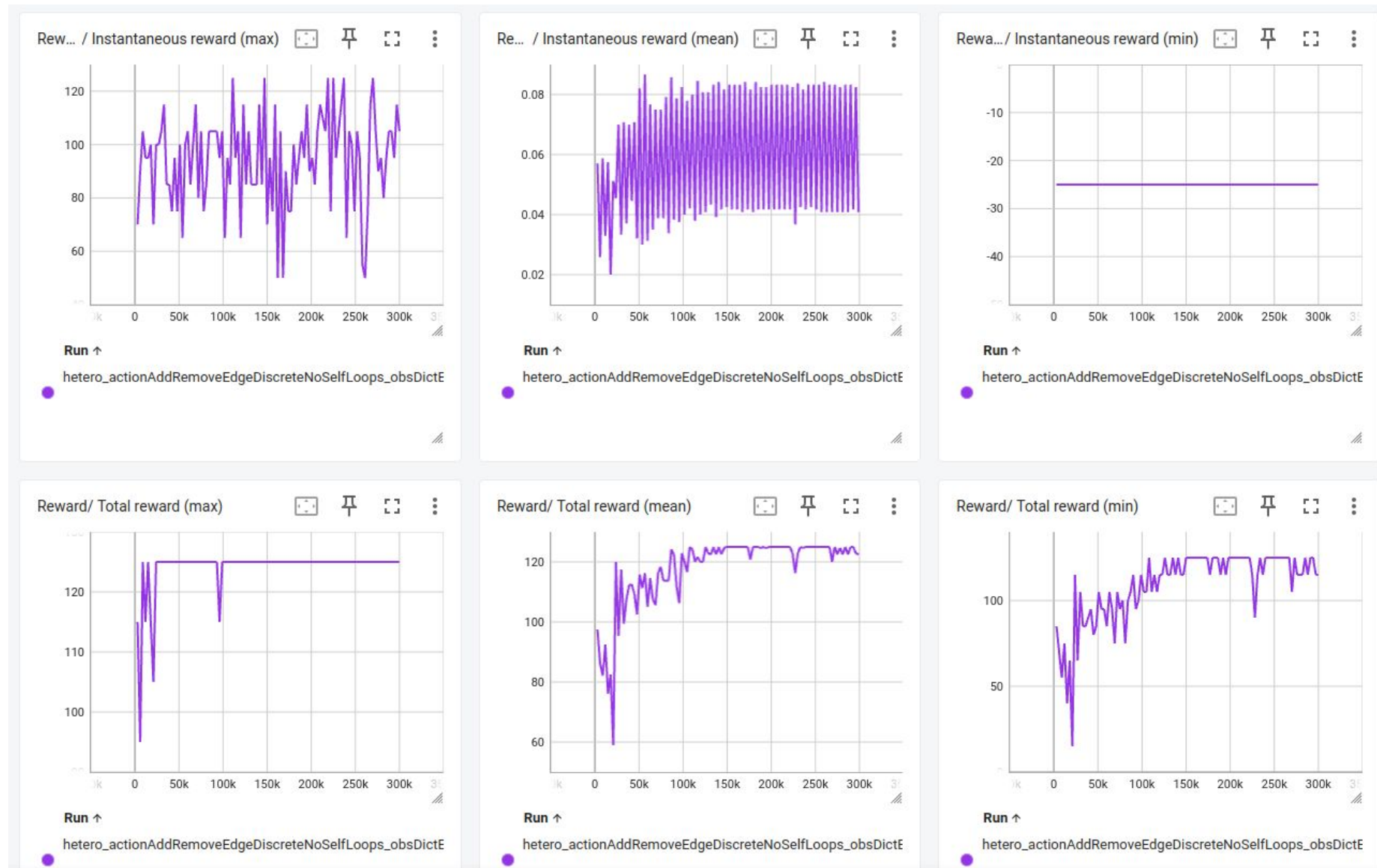


Trained on:

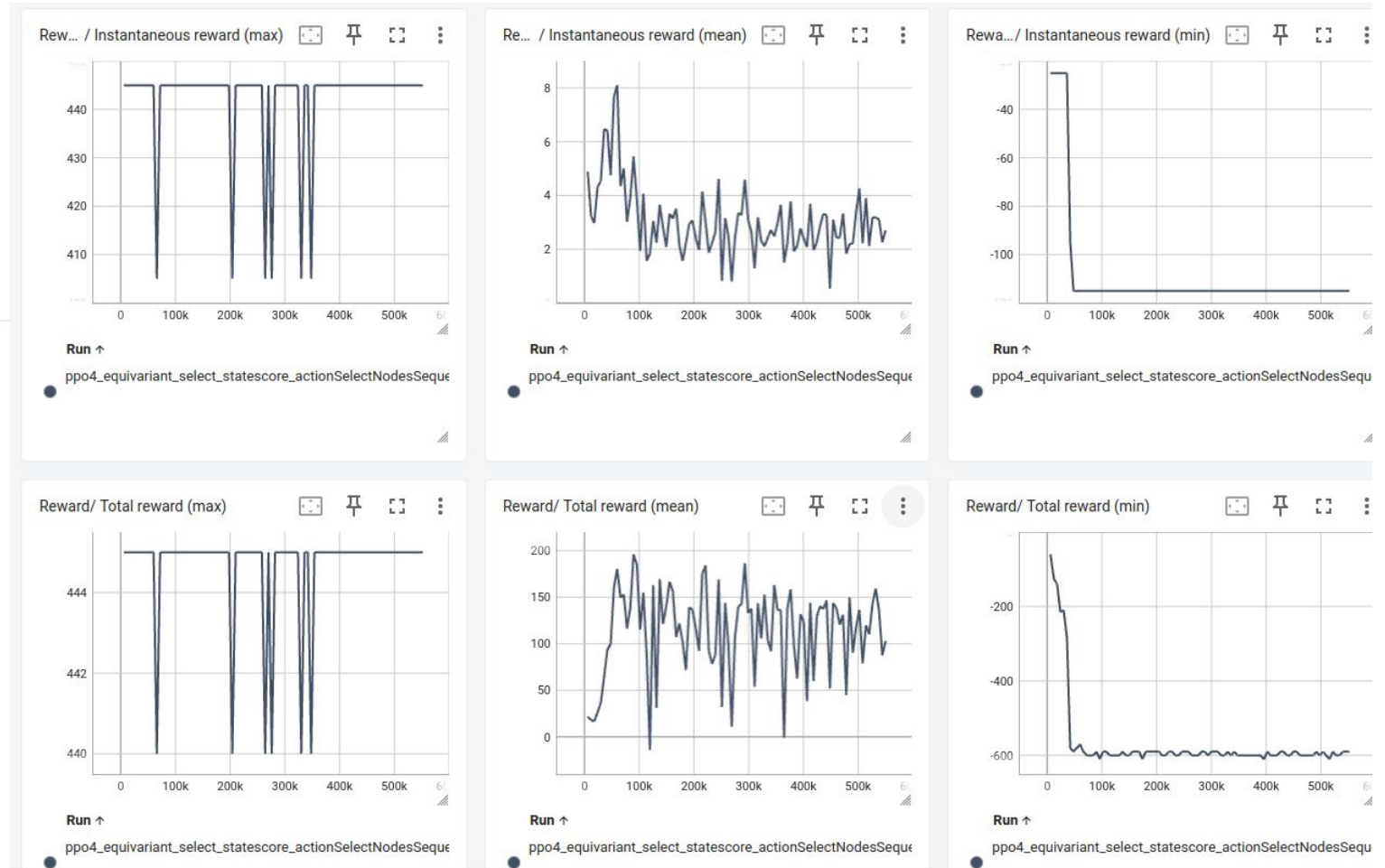
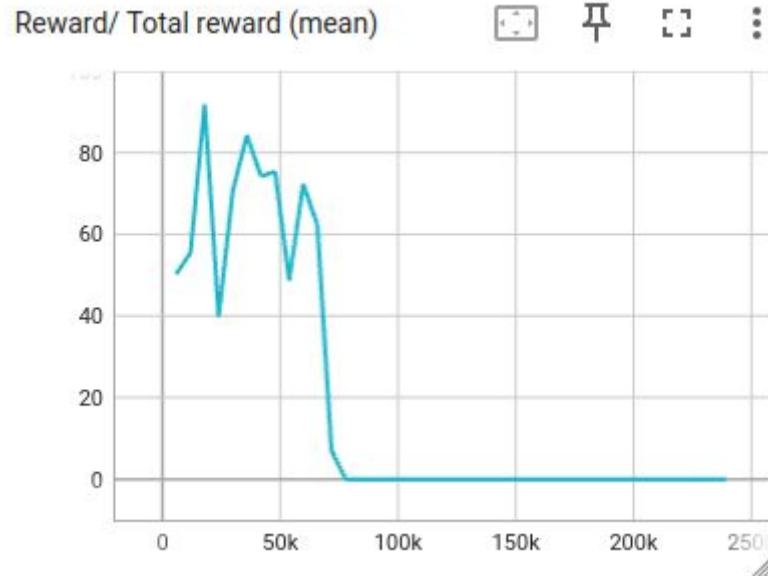
- 5 nodes
- 4 in R^2
- 1 in $SE(3)$
- “MaxSteps” termination

Model:

- DQN
- EGNN
- Add/Remove action



The figure below was from a DQN run with Select Node action type. The agent learned to select a node (which does not give a reward until the second selection) and then take the skip action to reset the selection.



Trained on:

- 8 nodes in R^3
- “MaxSteps” termination

Model:

- DQN
- EGNN
- Select Nodes action



Baseline comparison — selectseq-n8_R3

model : pposelect8_actionSelectNodesSequentially_obsDictEquivariantNodeFeaturesAndAdjAndSelection_n8_R3
environment : env_actionSelectNodesSequentially_obsDictEquivariantNodeFeaturesAndAdjAndSelection_rewardWeighted_termMaxSteps_n8_R3
network : 8 agents in R^3, action space SelectNodesSequentially
objective : Weighted state score
policy : PPO, --policy-mode sample, 200-step budget
measured on : 20 random networks · seed 0

method	edges fewer is better	score ϕ higher is better	rigid % of networks	minimal % of networks	margin mean $\pm$ sd, higher is better	margin (geo) gmean $\times/\div$ gsd	work edits applied	best at step reached
▪▪▪▪ initial	12.70 $\pm$ 12.06	127.00 $\pm$ 96.80	45	0	2.3e-05 $\pm$ 5.0e-05	1.0e-05 $\times/\div$ 14.4*	—	—
— random	14.55 $\pm$ 4.03	244.50 $\pm$ 33.39	70	0	8.4e-06 $\pm$ 1.4e-05	2.2e-06 $\times/\div$ 15.1*	71.3 $\pm$ 5.2	42.9 $\pm$ 47.7
— greedy	10.65 $\pm$ 0.57	293.50 $\pm$ 5.72	100	40	1.2e-06 $\pm$ 1.7e-06	2.3e-07 $\times/\div$ 8.6	11.2 $\pm$ 7.2	11.2 $\pm$ 7.2
— learned	10.60 $\pm$ 1.11	294.00 $\pm$ 11.14	100	70	1.1e-06 $\pm$ 1.6e-06	1.9e-07 $\times/\div$ 13.6	96.4 $\pm$ 2.5	79.3 $\pm$ 54.6

METHODS

- initial the random graph each method starts from
- random uniform random actions - the floor any method should beat
- greedy repeatedly applies the single best edge change until none helps
- learned the trained policy

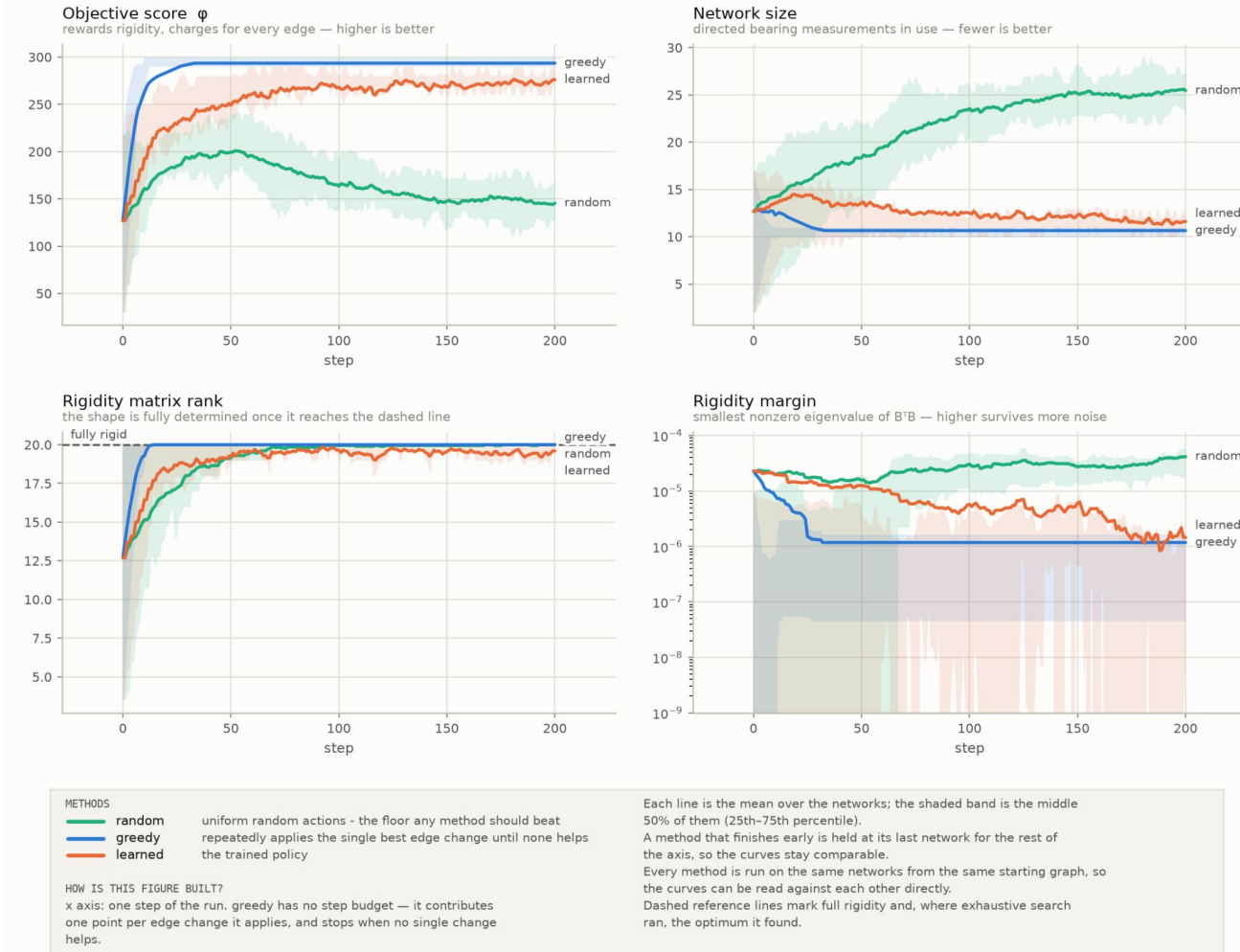
HOW IS THIS FIGURE BUILT?

edges: directed bearing measurements the network needs — each one is a sensing or communication link, so fewer is better.
score ϕ : the objective every method is scored with. It rewards rigidity and charges for each extra edge.
rigid: the network's shape is fully determined by its bearing measurements — the property being solved for. minimal: rigid with the fewest possible edges (a heuristic on mixed-domain networks, so it can under-report).

margin: how much room the network has before it stops being rigid. Its absolute size depends on how far apart the agents are, so compare rows rather than the number.
margin (geo): the same quantity as a geometric mean and spread, because it ranges over orders of magnitude — 'a $\times/\div$ b' means the typical network sits between a/b and a**. A '*' marks rows where non-rigid networks had to be left out: their margin is exactly 0, which no geometric mean can take, so those rows describe only the networks that came out rigid.
work: changes to the network the method actually applied. best at: the step its best network was reached — lower means it converged sooner and the rest of the budget added nothing.
initial and optimal are reference rows, not competing methods: every method starts from initial, and optimal is the best achievable. All methods see the same networks.
Every value is a mean over the networks and $\pm$ is the standard deviation across them. The percentage columns carry no $\pm$: they are already means of a yes/no outcome, whose spread is fixed by the percentage itself.**

Run trajectories — selectseq-n8_R3

model : pposelect8_actionSelectNodesSequentially_obsDictEquivariantNodeFeaturesAndAdjAndSelection_n8_R3
 environment : env_actionSelectNodesSequentially_obsDictEquivariantNodeFeaturesAndAdjAndSelection_rewardWeighted_termMaxSteps_n8_R3
 network : 8 agents in $\mathbb{R}^3$, action space SelectNodesSequentially
 measured on : 20 random networks · seed 0



Outcome across networks — selectseq-n8_R3

model : pposelect8_actionSelectNodesSequentially_obsDictEquivariantNodeFeaturesAndAdjAndSelection_n8_R3
 environment : env_actionSelectNodesSequentially_obsDictEquivariantNodeFeaturesAndAdjAndSelection_rewardWeighted_termMaxSteps_n8_R3
 network : 8 agents in R³, action space SelectNodesSequentially
 measured on : 20 random networks · seed 0

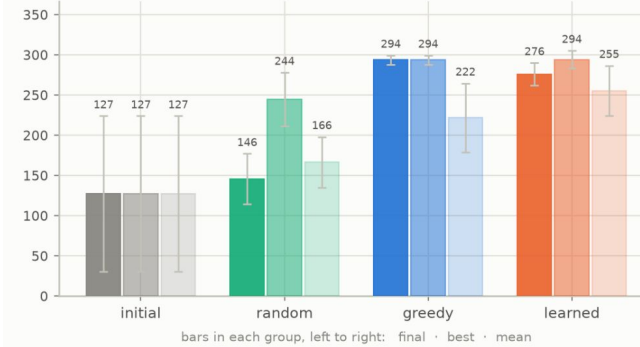


Final / best / mean outcome — selectseq-n8_R3

model : pposelect8_actionSelectNodesSequentially_obsDictEquivariantNodeFeaturesAndAdjAndSelection_n8_R3
 environment : env_actionSelectNodesSequentially_obsDictEquivariantNodeFeaturesAndAdjAndSelection_rewardWeighted_termMaxSteps_n8_R3
 network : 8 agents in R^3 , action space SelectNodesSequentially
 measured on : 20 random networks · seed 0

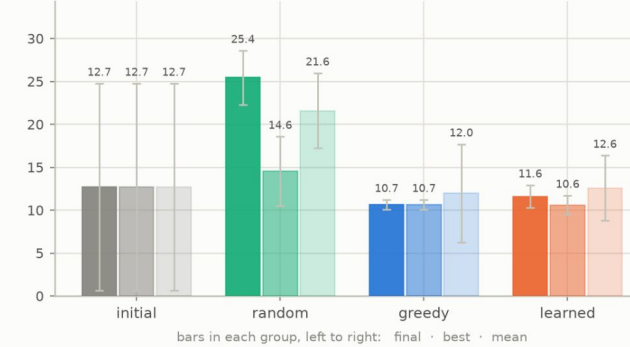
Objective score ϕ

rewards rigidity, charges for every edge — higher is better



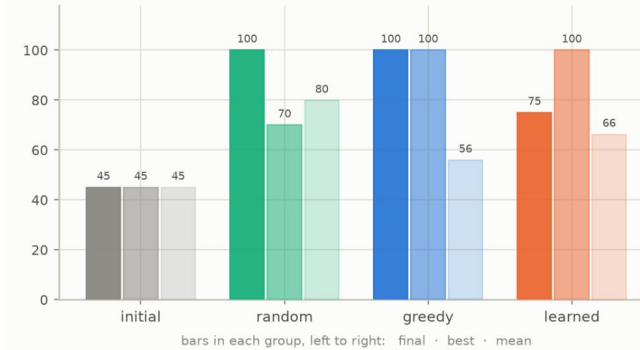
Network size

directed bearing measurements in use — fewer is better

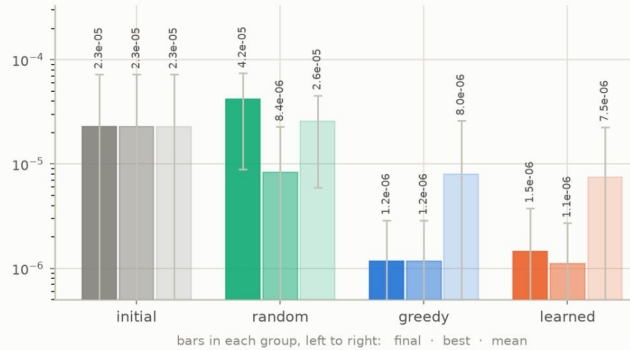


Rigidity achieved

% of networks (final, best) or % of the run spent rigid (mean)



Rigidity margin

smallest nonzero eigenvalue of B^*B — higher survives more noise

METHODS

- initial the random graph each method starts from
- random uniform random actions - the floor any method should beat
- greedy repeatedly applies the single best edge change until none helps
- learned the trained policy

HOW IS THIS FIGURE BUILT?

final: the network the run ended on

best: the best-scoring network the run passed through
 mean: averaged over every step of the run
 Bars are the mean over the networks; the whisker is ± 1 standard deviation across them.
 For a method that never moves (initial) or only improves (greedy), final and best are the same bar by construction.
 The rigidity margin is plotted on a log axis; a non-rigid network has margin 0 and cannot be drawn there.

Baseline comparison — addremove-hetero

```
model      : hetero_actionAddRemoveEdgeDiscreteNoSelfLoops_obsDictEquivariantNodeFeaturesAndAdjAndSelection_hetero
environment : env_actionAddRemoveEdgeDiscreteNoSelfLoops_obsDictEquivariantNodeFeaturesAndAdjAndSelection_rewardWeighted_termMaxSteps_hetero
network    : 5 agents in mixed ['R^2', 'SE(3)'], action space AddRemoveEdgeDiscreteNoSelfLoops
objective   : Weighted state score
policy     : DQN, --policy-mode sample, 50-step budget
measured on : 20 random networks · seed 0
```

method	edges	score ϕ	rigid	minimal	margin	margin (geo)	work	best at
	fewer is better	higher is better	% of networks	% of networks	mean $\pm$ sd, higher is better	gmean $\times/\div$ gsd	edits applied	step reached
▪▪▪▪ initial	7.25 $\pm$ 4.48	91.50 $\pm$ 50.13	20	0	1.5e-06 $\pm$ 4.9e-06	2.6e-06 $\times/\div$ 5.6*	—	—
— random	9.65 $\pm$ 1.65	162.50 $\pm$ 21.42	60	5	6.9e-07 $\pm$ 1.9e-06	1.8e-07 $\times/\div$ 10.4*	23.9 $\pm$ 3.0	27.4 $\pm$ 16.6
— greedy	9.00 $\pm$ 0.00	190.00 $\pm$ 0.00	100	55	3.5e-06 $\pm$ 6.5e-06	5.9e-07 $\times/\div$ 13.5*	5.2 $\pm$ 2.1	5.2 $\pm$ 2.1
— learned	9.00 $\pm$ 0.00	190.00 $\pm$ 0.00	100	60	2.9e-06 $\pm$ 4.6e-06	4.0e-07 $\times/\div$ 14.4	6.0 $\pm$ 2.4	5.2 $\pm$ 2.1

METHODS

▪▪▪▪ initial the random graph each method starts from

— random uniform random actions - the floor any method should beat

— greedy repeatedly applies the single best edge change until none helps

— learned the trained policy

HOW IS THIS FIGURE BUILT?

edges: directed bearing measurements the network needs — each one is a sensing or communication link, so fewer is better.

score ϕ : the objective every method is scored with. It rewards rigidity and charges for each extra edge.

rigid: the network's shape is fully determined by its bearing measurements — the property being solved for. minimal: rigid with the fewest possible edges (a heuristic on mixed-domain networks, so it can under-report).

margin: how much room the network has before it stops being rigid. Its absolute size depends on how far apart the agents are, so compare rows rather than the number.

margin (geo): the same quantity as a geometric mean and spread, because it ranges over orders of magnitude — 'a $\times/\div$ b' means the typical network sits between a/b and a**. A '*' marks rows where non-rigid networks had to be left out: their margin is exactly 0, which no geometric mean can take, so those rows describe only the networks that came out rigid.**

work: changes to the network the method actually applied. best at: the step its best network was reached — lower means it converged sooner and the rest of the budget added nothing.

initial and optimal are reference rows, not competing methods: every method starts from initial, and optimal is the best achievable. All methods see the same networks.

Every value is a mean over the networks and $\pm$ is the standard deviation across them. The percentage columns carry no $\pm$: they are already means of a yes/no outcome, whose spread is fixed by the percentage itself.

Baseline comparison — addremove-heteroComplex

```
model      : hetero_actionAddRemoveEdgeDiscreteNoSelfLoops_obsDictEquivariantNodeFeaturesAndAdjAndSelection_hetero
environment : env_actionAddRemoveEdgeDiscreteNoSelfLoops_obsDictEquivariantNodeFeaturesAndAdjAndSelection_rewardWeighted_termMaxSteps_heteroComplex
network    : 7 agents in mixed ['R^2', 'SE(3)'], action space AddRemoveEdgeDiscreteNoSelfLoops
objective   : Weighted state score
policy     : DQN, --policy-mode sample, 200-step budget
measured on : 20 random networks · seed 0
```

method	edges	score ϕ	rigid	minimal	margin	margin (geo)	work	best at
	fewer is better	higher is better	% of networks	% of networks	mean $\pm$ sd, higher is better	gmean $\times/\div$ gsd	edits applied	step reached
▪▪▪▪ initial	11.50 $\pm$ 8.44	198.00 $\pm$ 97.70	20	0	6.8e-06 $\pm$ 1.7e-05	2.6e-05 $\times/\div$ 2.1*	—	—
— random	15.65 $\pm$ 1.49	354.50 $\pm$ 12.84	60	5	7.8e-07 $\pm$ 1.7e-06	1.2e-07 $\times/\div$ 19.7*	98.0 $\pm$ 5.6	75.5 $\pm$ 51.0
— greedy	13.90 $\pm$ 0.62	381.00 $\pm$ 6.24	100	45	4.6e-07 $\pm$ 7.5e-07	1.1e-07 $\times/\div$ 9.7	9.4 $\pm$ 3.7	9.4 $\pm$ 3.7
— learned	15.45 $\pm$ 1.36	360.50 $\pm$ 11.61	85	5	3.1e-06 $\pm$ 3.4e-06	8.0e-07 $\times/\div$ 19.3*	118.2 $\pm$ 90.6	9.2 $\pm$ 4.6

METHODS

- initial the random graph each method starts from
- random uniform random actions - the floor any method should beat
- greedy repeatedly applies the single best edge change until none helps
- learned the trained policy

HOW IS THIS FIGURE BUILT?

edges: directed bearing measurements the network needs — each one is a sensing or communication link, so fewer is better.
score ϕ : the objective every method is scored with. It rewards rigidity and charges for each extra edge.
rigid: the network's shape is fully determined by its bearing measurements — the property being solved for. minimal: rigid with the fewest possible edges (a heuristic on mixed-domain networks, so it can under-report).

margin: how much room the network has before it stops being rigid. Its absolute size depends on how far apart the agents are, so compare rows rather than the number.
margin (geo): the same quantity as a geometric mean and spread, because it ranges over orders of magnitude — 'a $\times/\div$ b' means the typical network sits between a/b and a**. A '*' marks rows where non-rigid networks had to be left out: their margin is exactly 0, which no geometric mean can take, so those rows describe only the networks that came out rigid.
work: changes to the network the method actually applied. best at: the step its best network was reached — lower means it converged sooner and the rest of the budget added nothing.
initial and optimal are reference rows, not competing methods: every method starts from initial, and optimal is the best achievable. All methods see the same networks.
Every value is a mean over the networks and $\pm$ is the standard deviation across them. The percentage columns carry no $\pm$: they are already means of a yes/no outcome, whose spread is fixed by the percentage itself.**

Baseline comparison — selectseq-n8_R3

```

model      : bigDQN8SelectEquivariant3e-4lrNormalizedPositions
environment : env_actionSelectNodesSequentially_obsDictEquivariantNodeFeaturesAndAdjAndSelection_rewardWeighted_termMaxSteps_n8_R3
network    : 8 agents in R^3, action space SelectNodesSequentially
objective   : Weighted state score
policy     : DQN, --policy-mode sample, 200-step budget
measured on : 20 random networks · seed 0

```

method	edges fewer is better	score ϕ higher is better	rigid % of networks	minimal % of networks	margin mean $\pm$ sd, higher is better	margin (geo) gmean $\times/\div$ gsd	work edits applied	best at step reached
▪▪▪▪ initial	12.70 $\pm$ 12.06	127.00 $\pm$ 96.80	45	0	2.3e-01 $\pm$ 5.0e-01	1.0e-01 $\times/\div$ 14.4*	—	—
— random	14.55 $\pm$ 4.03	244.50 $\pm$ 33.39	70	0	8.4e-02 $\pm$ 1.4e-01	2.2e-02 $\times/\div$ 15.1*	71.3 $\pm$ 5.2	42.9 $\pm$ 47.7
— greedy	10.65 $\pm$ 0.57	293.50 $\pm$ 5.72	100	40	1.2e-02 $\pm$ 1.7e-02	2.3e-03 $\times/\div$ 8.6	11.2 $\pm$ 7.2	11.2 $\pm$ 7.2
— learned	12.95 $\pm$ 7.10	270.50 $\pm$ 70.96	100	75	1.3e-01 $\pm$ 3.4e-01	2.3e-03 $\times/\div$ 19.6	100.0 $\pm$ 0.0	15.5 $\pm$ 6.7

METHODS

▪▪▪▪ initial the random graph each method starts from
 — random uniform random actions - the floor any method should beat
 — greedy repeatedly applies the single best edge change until none helps
 — learned the trained policy

HOW IS THIS FIGURE BUILT?

edges: directed bearing measurements the network needs — each one is a sensing or communication link, so fewer is better.

score ϕ : the objective every method is scored with. It rewards rigidity and charges for each extra edge.

rigid: the network's shape is fully determined by its bearing measurements — the property being solved for. minimal: rigid with the fewest possible edges (a heuristic on mixed-domain networks, so it can under-report).

margin: how much room the network has before it stops being rigid. Its absolute size depends on how far apart the agents are, so compare rows rather than the number.

margin (geo): the same quantity as a geometric mean and spread, because it ranges over orders of magnitude — 'a $\times/\div$ b' means the typical network sits between a/b and a·b. A '*' marks rows where non-rigid networks had to be left out: their margin is exactly 0, which no geometric mean can take, so those rows describe only the networks that came out rigid.

work: changes to the network the method actually applied. best at: the step its best network was reached — lower means it converged sooner and the rest of the budget added nothing.

initial and optimal are reference rows, not competing methods: every method starts from initial, and optimal is the best achievable. All methods see the same networks.

Every value is a mean over the networks and $\pm$ is the standard deviation across them. The percentage columns carry no $\pm$: they are already means of a yes/no outcome, whose spread is fixed by the percentage itself.


```
#####
# ACTION_TYPE = "AllEdges"
# ACTION_TYPE = "AddRemoveEdgeMultiDiscrete"
# ACTION_TYPE = "AddRemoveEdgeDiscrete"
# ACTION_TYPE = "AddEdgeDiscrete"
# ACTION_TYPE = "AddEdgeDiscreteNoSkip"
# ACTION_TYPE = "AddEdgeDiscreteNoSelfLoops"
# ACTION_TYPE = "AddEdgeDiscreteNoSkipNoSelfLoops"
# ACTION_TYPE = "AddRemoveEdgeDiscreteNoSelfLoops"
ACTION_TYPE = "SelectNodesSequentially"
# ACTION_TYPE = "DecideOnEdge"

# ACTION_REWARDS_ENABLE = True
ACTION_REWARDS_ENABLE = False

TIME_PENALTY_VALUE = 5.0

SKIP_IS_STOP = True
RANDOM_GRAPH_WITH_MEAN_MIN_EDGES = False

# INCREMENTAL_REWARDS_ENABLE = False
INCREMENTAL_REWARDS_ENABLE = True

TRACK_DATA_ENABLE = True
# TRACK_DATA_ENABLE = False

# OBS_TYPE = "Complete"
# OBS_TYPE = "CompleteAndEigenvalues"
# OBS_TYPE = "AdjFlatAndEigenvalues"
# OBS_TYPE = "DictNodeFeaturesAndAdj"
# OBS_TYPE = "DictNodeFeaturesAndAdjAndSelection"
# OBS_TYPE = "DictNodeFeaturesAndAdjAndEdgeProposal"
OBS_TYPE = "DictEquivariantNodeFeaturesAndAdjAndSelection" ## Equivariant
# OBS_TYPE = "DictBearingNodeFeaturesAndAdjAndSelection"
# OBS_TYPE = "DictNodeFeaturesAndEdgeFeaturesAndAdjAndSelection" ## GINE
```

```
# STATE_SCORE_TYPE = "Rigid"
# STATE_SCORE_TYPE = "RigidAndMinEigenvalue"
# STATE_SCORE_TYPE = "RigidAndMinRigid"
# STATE_SCORE_TYPE = "RigidAndLogMinEigenvalueAndEdges"
# STATE_SCORE_TYPE = "MinRigid"
# STATE_SCORE_TYPE = "MinRigidAndMinEigenvalue"
# STATE_SCORE_TYPE = "MinEigenvalue"
# STATE_SCORE_TYPE = "Eigenvalues"
# STATE_SCORE_TYPE = "EdgeCount"
# STATE_SCORE_TYPE = "LogMinEigenvalue"
# STATE_SCORE_TYPE = "RigidityMatrixRank"
# STATE_SCORE_TYPE = "RigidityMatrixRankAndEdges"
STATE_SCORE_TYPE = "Weighted"
# STATE_SCORE_TYPE = "None"

# TERMINATION_CONDITION_TYPE = "MaxSteps"
# TERMINATION_CONDITION_TYPE = "MaxStepsRankBonus"
# TERMINATION_CONDITION_TYPE = "Rigid"
# TERMINATION_CONDITION_TYPE = "RigidMinEigBonus"
TERMINATION_CONDITION_TYPE = "MinimallyRigid"
# TERMINATION_CONDITION_TYPE = "RigidMinEigAndEdgesBonus"
# TERMINATION_CONDITION_TYPE = "Bandit"

MAX_STEPS = 2000

TRUNCATE_ENABLE = True
TRUNCATE_MAX_STEPS = 100
TRUNCATE_PENALTY_VALUE = 100

ONLY_RANDOMIZE_EDGES = False
#####
```

DQN Training Configuration

```
#####
TOTAL_TIMESTEPS = int(3e5)
NR_ENVS = 4
MEM_SIZE = 20000
EGREEDY_STEPS = 150000

GNN_HIDDEN_DIM = 128
QNETWORK_HEAD_HIDDEN_DIM = 128

cfg = DQN_CFG()
cfg.experiment.directory = "runs"
cfg.batch_size = 128
cfg.target_update_interval = 1000
cfg.update_interval = 4
cfg.learning_starts = MEM_SIZE + 1
cfg.discount_factor = 0.99
# cfg.random_timesteps = MEM_SIZE

def epsilon_schedule(timestep, timesteps):
    start = 0.8
    end = 0.05
    decay_steps = min(EGREEDY_STEPS, timesteps)
    eps = start - (start - end) * min(1.0, timestep / decay_steps)
    return eps
cfg.exploration_scheduler = epsilon_schedule

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
#####
```

PPO Training Configuration

```
#####
TOTAL_TIMESTEPS = int(1e6)
NR_ENVS = 1
MEM_SIZE = 4092 * 2

GNN_HIDDEN_DIM = 128
ACTOR_HEAD_HIDDEN_DIM = 256
CRITIC_HEAD_HIDDEN_DIM = 256

cfg = PPO_CFG()
cfg.rollouts = MEM_SIZE # to ensure we don't get garbage data from memory
cfg.experiment.directory = "runs"
# incentivize exploration more
cfg.entropy_loss_scale = 0.01

DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
#####
```

Graph Reinforcement Learning for Combinatorial Optimization

V.-A. Darvari, S. Hailes, and M. Musolesi, "Graph Reinforcement Learning for Combinatorial Optimization: A Survey and Unifying Perspective," Aug. 20, 2024, arXiv: arXiv:2404.06492. doi: 10.48550/arXiv.2404.06492.

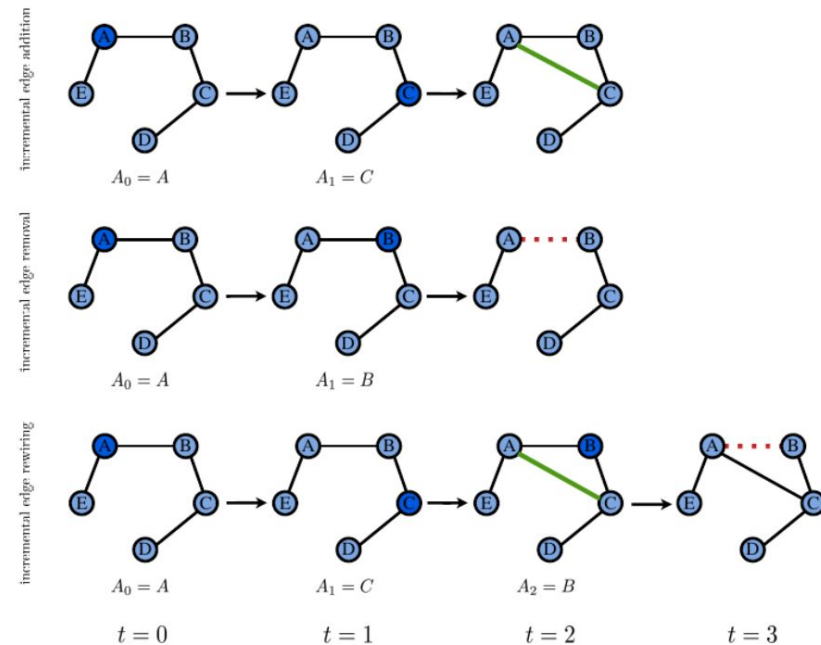
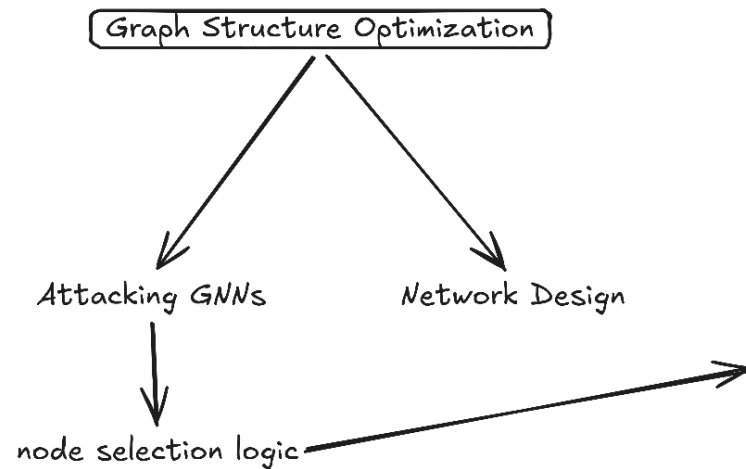


Figure 4: Illustration of several action space designs for *Graph Structure Optimization*. Edge addition, removal, and rewiring can be formulated as the selection of a single node per timestep, yielding an $\mathcal{O}(|V|)$ action space. The topological changes are encapsulated in the definition of the transition function. Constraints are commonly used to exclude invalid actions (e.g., the actions corresponding to the addition of an edge that is already part of the graph will be forbidden).

Also:

- Edge Rewiring
- Using GIN GNNs.
- Action masking.
- Multi-Objective reward.
- Only termination reward.

- **Questions:** Is there a conceptual problem in the environment design or training approach? Does reward function need tuning? What about observation? Do training hyperparameters need tuning? Is there an architectural problem?
 - **Learning the “skip/stop” action is hard** since it is (ideally) a good action only in one state.
 - A node’s rigidity (“locked” DOFs) depend on its domain and number of bearing measurements. The **target node’s features do not matter for the rigidity of the base node**, however its rigidity might. Then does it make sense to use **message passing**?
 - For training, random networks are generated (random number of edges). However, as the **number of nodes grow**, the **minimum number of edges** that can make the network IBR **doesn’t grow as fast**. This makes it difficult for the RL agent to train on high number of nodes.
 - Using **Minimal Bearing Rigidity (MBR)** on heterogeneous networks with R^d domain is quite helpful, mainly as a termination condition we can use during training. However there is **no closed form formula** for other domains or heterogeneous networks. Can we **count the DOFs edges constrain**? However not only the degrees of freedom vary per node, but the unobservable null space of the formation shifts depending on exactly which domains are interacting. Also, a node may not need a certain number of outgoing edges depending on the rigidity of the whole graph, so **we can’t count the edges needed per node**.
 - Maybe a better approach would be to **let the RL agent figure out a graph configuration that is minimally rigid (but not optimal)**, then consider another problem that is **rewiring the current edges in the graph for the spectral properties**.
 - What about **pre-training the GNN for rigidity** and using **RL for the action head**?
-

- **Rigidity:** Investigate Eigenvalues in the state score.
 - **Performance metrics and baselines:** Greedy hill-climbing, Brute force, Neural-Combinatorial-Optimization?
 - **Training Duration:** Using a custom gymnasium environment currently, however the matrix rank, eigenvalue (SVD), centrality features etc. are quite expensive and training is slow. I am currently trying to rewrite the environment as a tensorized pytorch environment that can run on CUDA.
 - **Approach Idea:** Start with an empty graph and only add edges. Terminate when rigidity is reached and give a termination reward based on the state's score (we lose the "repairability"). Not a good configuration space searcher.
 - **Approach Idea:** Train two RL agents, one for construction and one for pruning. Let them run iteratively until they both converge (take a "done" action maybe).
 - **MARL:** I would like to experiment with a CTDE multi-agent scheme. The actor network can see a k-hop neighborhood of the node, and the value network can stay the same as current one. What should k be? Does it make sense, and how would we go about this?
 - **Limited FOV:** We can experiment with agents in the network having limited bearing measurement capabilities. Currently any agent can measure their bearing to every other one. (Limited FOV would be simply an action mask)
-